

PXDesign on Blackwell (RTX 50-series)

Complete replication guide — Ubuntu + RTX 5070 Ti (sm_120) + uv
Built and verified on IMILGPU2, 7 October 2026. Source machine: `/data/imil/envs/pxdesign-venv`

Why this document exists. PXDesign's official `install.sh` pins `torch==2.3.1` with `cu121 / cu124` wheels and `jax[cuda]==0.4.29`. Neither supports compute capability `sm_120`. Blackwell GPUs require PyTorch built with CUDA 12.8 or newer; with an older build every kernel launch fails with CUDA error: no kernel image is available for execution on the device. This guide installs a modernised stack instead, then patches the four API breakages that modernisation causes. Following the upstream installer on a 50-series card will not work.

1. Target system

Component	Value on source machine	Requirement
GPU	NVIDIA RTX 5070 Ti, 16 GB, sm_120	Any Blackwell card
Driver	595.91.07 (CUDA 13.2)	Must support CUDA 12.8+
OS	Ubuntu (glibc 2.39, GCC 13.3)	Linux x86_64
Python	3.12.3	3.12 recommended (see note)
Package manager	uv 0.12.19	uv or pip
Peak VRAM observed	6.6 GB of 16 GB (546 tokens, preview mode)	≥12 GB advised
Disk for software + weights	~12 GB	≥20 GB free

Why Python 3.12. Torch wheels are built per Python version (`cp312`). Using 3.12 lets uv reuse a cached `torch-2.11.0+cu128` wheel if one is already present. PXDesign's `setup.py` only requires `>=3.10`, so 3.11 also works — it just costs a fresh multi-GB download.

2. Prepare the uv cache (optional but strongly advised)

uv hardlinks from its cache into a venv only when both sit on the same filesystem; otherwise it silently falls back to full copies. If your work lives on a data disk, move the cache there. This turns multi-GB installs into near-instant hardlinks.

```
# one-time: put the cache on the same filesystem as your venvs
mv ~/.cache/uv /data/imil/.uv-cache          # adjust paths to your machine
mkdir -p ~/.config/uv
cat > ~/.config/uv/uv.toml <<'TOML'
cache-dir = "/data/imil/.uv-cache"
link-mode = "hardlink"
TOML

uv cache dir                                # confirm
stat -c '%d %n' /data/imil/.uv-cache /data/imil/envs # device IDs must match
```

3. Clone the repository

PXDesign declares one submodule (`eval_design` → `PXDesignBench`), but the installed `pxdbench` package comes from PyPI-style git install, not the submodule. Cloning with `--recurse-submodules` is harmless either way.

```
mkdir -p /data/imil/TEST_SPACE/BINDER_DESIGN/software
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software
git clone --recurse-submodules https://github.com/bytedance/PXDesign.git
cd PXDesign
```

Do not run `install.sh`. It is conda-only and installs the incompatible torch 2.3.1. Everything below replaces it.

4. Create the venv and install PyTorch

```
uv venv /data/imil/envs/pxdesign-venv --python 3.12 --relocatable
export PXPY=/data/imil/envs/pxdesign-venv/bin/python

uv pip install --python $PXPY \
  --index-url https://download.pytorch.org/whl/cu128 \
  torch==2.11.0
```

Verify `sm_120` before continuing. A capability check alone is not enough — you need a real kernel launch:

```
$PXPY -c '
import torch
print(torch.__version__, torch.version.cuda)
print(torch.cuda.get_device_name(0), torch.cuda.get_device_capability(0))
x = torch.randn(4096, 4096, device="cuda", dtype=torch.bfloat16)
print("matmul ok:", float((x @ x).float().abs().mean()))'
```

Expected. `2.11.0+cu128 12.8 / NVIDIA GeForce RTX 5070 Ti (12, 0) / a finite number`. If you see "no kernel image is available", stop — nothing downstream can work.

5. Install dependencies in order

`--no-deps` on the git installs is **mandatory**. Protenix pins `torch==2.3.1` ; without the flag `uv` will downgrade your working torch and silently undo section 4.

5a. PXDesign's own Python dependencies

Taken from `requirements.txt` , minus `torch==2.3.1` , `numpy==1.26.3` , `nvidia-cublas-cu12` (torch supplies these), and `deepspeed` (optional, see §9). `click` is *missing* from upstream requirements but required by the CLI.

```
uv pip install --python $PXPY \
  click PyYaml scipy ml_collections tqdm pandas optree rdkit \
  biopython==1.83 modelcif==0.7 biotite==1.0.1 \
  scikit-learn scikit-learn-extra icecream natsort dm-tree \
  einops posix_ipc fair-esm matplotlib ipywidgets py3Dmol
```

5b. PXDesign itself (editable)

```
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign
uv pip install -e . --python $PXPY
```

Note: `install_requires` is commented out in `setup.py` , so this installs no dependencies — §5a does that work.

5c. Protenix and PXDesignBench

```
uv pip install --python $PXPY --no-deps \
  "git+https://github.com/bytedance/Protenix.git@v0.5.0+pxd"

uv pip install --python $PXPY --no-deps \
  "git+https://github.com/bytedance/PXDesignBench.git@v0.1.2"
```

5d. JAX — cu12 family only

Never mix cu12 and cu13 wheel families in one venv. Torch cu128 pulls `nvidia-*-cu12` wheels. JAX must come from the cu12 line or the two will fight over the same CUDA libraries and break each other.

```
uv pip install --python $PXPY "jax[cuda12]"

# verify BOTH frameworks still work
$PXPY -c 'import jax; print(jax.__version__, jax.devices())'
$PXPY -c 'import torch; x=torch.randn(8,8,device="cuda"); print("torch survived jax")'
```

Source machine resolved to jax 0.11.2 with `jax-cuda12-plugin` / `jax-cuda12-pjrt` 0.11.2.

5e. ColabDesign and the jax ecosystem

Versions here are deliberately newer than upstream's `dm-haiku==0.0.13` / `optax==0.2.5` , which belong to the jax 0.4.29 era.

```
uv pip install --python $PXPY --no-deps \
  "git+https://github.com/sokrypton/ColabDesign.git"

uv pip install --python $PXPY dm-haiku==0.0.17 optax==0.2.8 jmp
uv pip install --python $PXPY transformers==4.51.3
```

5f. Confirm the import graph

```
$PXPY -c 'import protenix, pxdbench, colabdesign, pxdesign; print("all four import")'
```

6. Patches — the part you cannot skip

Modernising jax and numpy breaks four call sites. All four are plumbing, none affect the science. Apply them all before running anything.

Patch 1 — `jax.lib.xla_bridge` removed

FILE `colabdesign/shared/utils.py` **SYMPTOM** `AttributeError: module 'jax.lib' has no attribute 'xla_bridge'` during the ProteinMPNN step.

```
SP=/data/imil/envs/pxdesign-venv/lib/python3.12/site-packages
cp $SP/colabdesign/shared/utils.py{,.bak}

python - <<'PY'
p = "/data/imil/envs/pxdesign-venv/lib/python3.12/site-packages/colabdesign/shared/utils.py"
old = """    backend = jax.lib.xla_bridge.get_backend()
    if hasattr(backend, 'live_buffers'):
        for buf in backend.live_buffers():
            buf.delete()"""
new = """    backend = None
    for _get in (lambda: __import__('jax.extend.backend', fromlist=['x']).get_backend(),
                lambda: jax.lib.xla_bridge.get_backend()):
        try:
            backend = _get(); break
        except Exception:
            pass
    if backend is not None and hasattr(backend, 'live_buffers'):
        try:
            for buf in backend.live_buffers():
                buf.delete()
        except Exception:
            pass"""
s = open(p).read()
assert old in s, "pattern not found - file differs from expected"
open(p,"w").write(s.replace(old, new))
print("patched")
PY

$PXPY -c "from colabdesign.shared.utils import clear_mem; clear_mem(); print('clear_mem ok')"
```

`clear_mem()` only frees GPU buffers between runs. On modern jax it degrades to `gc.collect()`, which is safe because `XLA_PYTHON_CLIENT_ALLOCATOR=platform` (\$8) returns memory promptly anyway.

Patch 2 — `jnp.clip(a_min=, a_max=)` renamed

FILES `ColabDesign's vendored AlphaFold` **SYMPTOM** `TypeError: clip() got an unexpected keyword argument 'a_max'` in the AF2 forward pass.

```
cd /data/imil/envs/pxdesign-venv/lib/python3.12/site-packages/colabdesign
grep -rl "a_max=\|a_min=" . --include='*.py' | while read f; do cp "$f" "$f.bak"; done
grep -rl "a_max=\|a_min=" . --include='*.py' | xargs sed -i 's/\ba_min=/min=/g; s/\ba_max=/max=/g'
grep -rn "a_max=\|a_min=" . --include='*.py' # must print nothing

# check pxdbench too
cd ../pxdbench && grep -rn "a_max=\|a_min=" . --include='*.py'
```

Patch 3 — `numpy 2.x float32` not JSON serializable

FILES `pxdbench/tools/af2/main_af2_complex.py`, `pxdbench/tools/af2/main_af2_monomer.py` **SYMPTOM** `TypeError: Object of type float32 is not JSON serializable`, followed by `JSONDecodeError` in the parent process. The sweep below finds every affected file.

```
python - <<'PY'
import os, glob
HELP = ''

class _NpEncoder(json.JSONEncoder):
    def default(self, o):
        import numpy as _np
        if isinstance(o, _np.integer): return int(o)
        if isinstance(o, _np.floating): return float(o)
        if isinstance(o, _np.bool_): return bool(o)
        if isinstance(o, _np.ndarray): return o.tolist()
        return super().default(o)

root = "/data/imil/envs/pxdesign-venv/lib/python3.12/site-packages/pxdbench"
for p in glob.glob(root+"/**/*.py", recursive=True):
    s = open(p).read()
    if "json.dump(results, f)" not in s: continue
    if "_NpEncoder" in s: print("already patched:", p); continue
    if not os.path.exists(p+".bak"): open(p+".bak","w").write(s)
    lines = s.split("\n")
    li = max(i for i,l in enumerate(lines[:60]) if l.startswith(("import ","from ")))
    lines.insert(li+1, HELP)
    s = "\n".join(lines).replace("json.dump(results, f)", "json.dump(results, f, cls=_NpEncoder)")
    open(p,"w").write(s)
    print("patched:", p)
PY
```

Patch 4 — not a patch: `--use_fast_ln` needs ninja

SYMPTOM `RuntimeError: Ninja is required to load C++ extensions when --use_fast_ln True`.

Protenix JIT-compiles a fused LayerNorm CUDA extension on import. Two options:

- **Run with** `--use_fast_ln False` (verified working on the source machine). Zero risk.
- **Or** `uv pip install --python $PXPY ninja` and test `--use_fast_ln True`. The JIT then compiles against `sm_120`, which is unverified. Allow one 5-minute test run to find out.

Record your patches

```
cd /data/imil/envs/pxdesign-venv/lib/python3.12/site-packages
for f in $(find colabdesign pxdbench -name '*.py.bak'); do diff -u "$f" "${f%.bak}"; done \
> /data/imil/envs/pxdesign-venv-PATCHES.diff
uv pip freeze --python $XPY > /data/imil/envs/pxdesign-venv-WORKING.txt
```

7. Model weights

```
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign
bash download_tool_weights.sh
```

Writes to `$(project_root)/tool_weights` and `$(project_root)/release_data/ccd_cache`. Verify, because the script reports "already exists — skipping" on directory presence alone:

```
du -sh tool_weights/af2 tool_weights/mpnn release_data/ccd_cache
```

Path	Expected size	Contents
tool_weights/af2	5.3 G	params_model_1-5 .npz, _ptm, _multimer_v3
tool_weights/mpnn	71 M	vanilla / soluble / ca model weights
release_data/ccd_cache	549 M	components.v20240608.cif (408 M), rdkit_mol.pkl (121 M), clusters (21 M)

Four PXDesign/Protenix checkpoints (~4 GB) download automatically into `release_data/checkpoint/` on first `infer` run: `pxdesign_v0.1.0.pt`, `protenix_base_default_v0.5.0.pt`, `protenix_mini_default_v0.5.0.pt`, `protenix_mini_tmpl_v0.5.0.pt`.

8. Environment variables

```
export PROTENIX_DATA_ROOT_DIR=/data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign/release_data/ccd_cache
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_ALLOCATOR=platform
```

The two XLA variables matter whenever `jax` and `torch` share the GPU in one process (preview and extended modes). By default `jax` grabs ~75 % of VRAM on import, starving `torch`.

Always activate the venv. `pxdbench` spawns ProteinMPNN and AF2 as *subprocesses* that inherit `PATH`. Calling the binary by absolute path without activating produces `ModuleNotFoundError: No module named 'colabdesign'` inside the subprocess.

```
source /data/imil/envs/pxdesign-venv/bin/activate
which python # must print /data/imil/envs/pxdesign-venv/bin/python
```

9. Dependencies you do NOT need

Package	Status	Why
deepspeed	Skip	Guarded by <code>importlib.util.find_spec</code> . Only serves <code>--use_deepspeed_evo_attention</code> , default False. Needs <code>nvcc</code> + <code>CUTLASS 3.5.1</code> .
flash_attn	Skip	Guarded. One of several Protenix attention backends.
torch_scatter	Skip	Function-body import in <code>protenix/utls/scatter_utils.py</code> with a fallback path. No prebuilt wheels for <code>torch 2.11+cu128</code> .
numpy==1.26.3	Do not pin	<code>pandas 3.x</code> and <code>scipy 1.18</code> require <code>numpy 2.x</code> . Patch 3 handles the consequence.
server_constant	Unavailable	ByteDance-internal, imported only by <code>pxdesign/pxd_server/</code> . Not on the CLI path.

10. End-to-end verification

```
source /data/imil/envs/pxdesign-venv/bin/activate
cd /data/imil/TEST_SPACE/BINDER_DESIGN/software/PXDesign
export PROTENIX_DATA_ROOT_DIR=$PWD/release_data/ccd_cache
export XLA_PYTHON_CLIENT_PREALLOCATE=false
export XLA_PYTHON_CLIENT_ALLOCATOR=platform

# 1. CLI responds
pxdesign --help

# 2. YAML validation (CPU, instant)
pxdesign check-input --yaml ./examples/PDL1_quick_start.yaml

# 3. Structure parsing (CPU, needs CCD cache)
pxdesign parse-target --yaml ./examples/PDL1_quick_start.yaml -o /tmp/px_debug

# 4. Diffusion only (GPU, ~15 s for 2 samples at 196 tokens)
pxdesign infer -i ./examples/PDL1_quick_start.yaml -o /tmp/px_smoke --N_sample 2

# 5. Full preview pipeline (GPU, ~7 min for 10 samples at 546 tokens)
pxdesign pipeline --preset preview -i ./examples/PDL1_quick_start.yaml \
-o /tmp/px_preview --N_sample 10 \
--dtype bf16 --use_fast_ln False --use_deepspeed_evo_attention False

find /tmp/px_preview -name summary.csv -exec head -3 {} \;
```

Success criterion. Step 5 must produce a `summary.csv` with populated `af2_*` columns and `AF2-IG-success` / `AF2-IG-easy-success` flags. Zero passing designs at `N=10` is normal; a missing or empty file is not.

11. Failure → cause reference

Error	Cause	Fix
no kernel image is available	torch built for CUDA <12.8	Reinstall torch from cu128 index (§4)
No module named 'click'	Missing from upstream requirements.txt	§5a
No module named 'protenix' / 'pxdbench'	Not yet installed — both are floor dependencies, needed even by check-input	§5c
No module named 'colabdesign' <i>inside a subprocess</i>	venv not activated; subprocess used system python	source ../activate (§8)
jax.lib has no attribute 'xla_bridge'	jax API removal	Patch 1
clip() got an unexpected keyword 'a_max'	jnp.clip signature change	Patch 2
Object of type float32 is not JSON serializable	numpy 2.x	Patch 3
Ninja is required to load C++ extensions	--use_fast_ln True JIT	Patch 4 — use False, or install ninja
Failed to open libtpu.so	jax probing for a TPU	Harmless, ignore
Delay kernel timed out (cuda_timer)	Profiling warmup warning	Harmless, ignore
PyMOL: No module named 'imp'	System PyMOL predates Python 3.12	Use a conda PyMOL or Mol* web viewer. Not needed by PXDesign.

12. Verified working version set

Package	Version	Note
python	3.12.3	
torch	2.11.0+cu128	upstream pins 2.3.1 — incompatible
jax / jaxlib	0.11.2 (cuda12 plugin)	upstream pins 0.4.29 — incompatible
numpy	2.5.3	upstream pins 1.26.3
dm-haiku	0.0.17	upstream pins 0.0.13
optax	0.2.8	upstream pins 0.2.5
transformers	4.51.3	as upstream
biopython / biotite / modelcif	1.83 / 1.0.1 / 0.7	as upstream
protenix	v0.5.0+pxd (git)	--no-deps
pxdbench	0.1.2 (git)	--no-deps
colabdesign	1.1.3 (git HEAD)	--no-deps , patched
pxdesign	0.1.0 (editable)	

The authoritative record is `pxdesign-venv-WORKING.txt` (full freeze) plus `pxdesign-venv-PATCHES.diff` (source edits) from §6. Copy both to the new machine and diff after installation.